Scientific Conference Organization Tool

DA – Turma 13

- Francisca Silveira
- Tiago Botelho
- Tiago Su

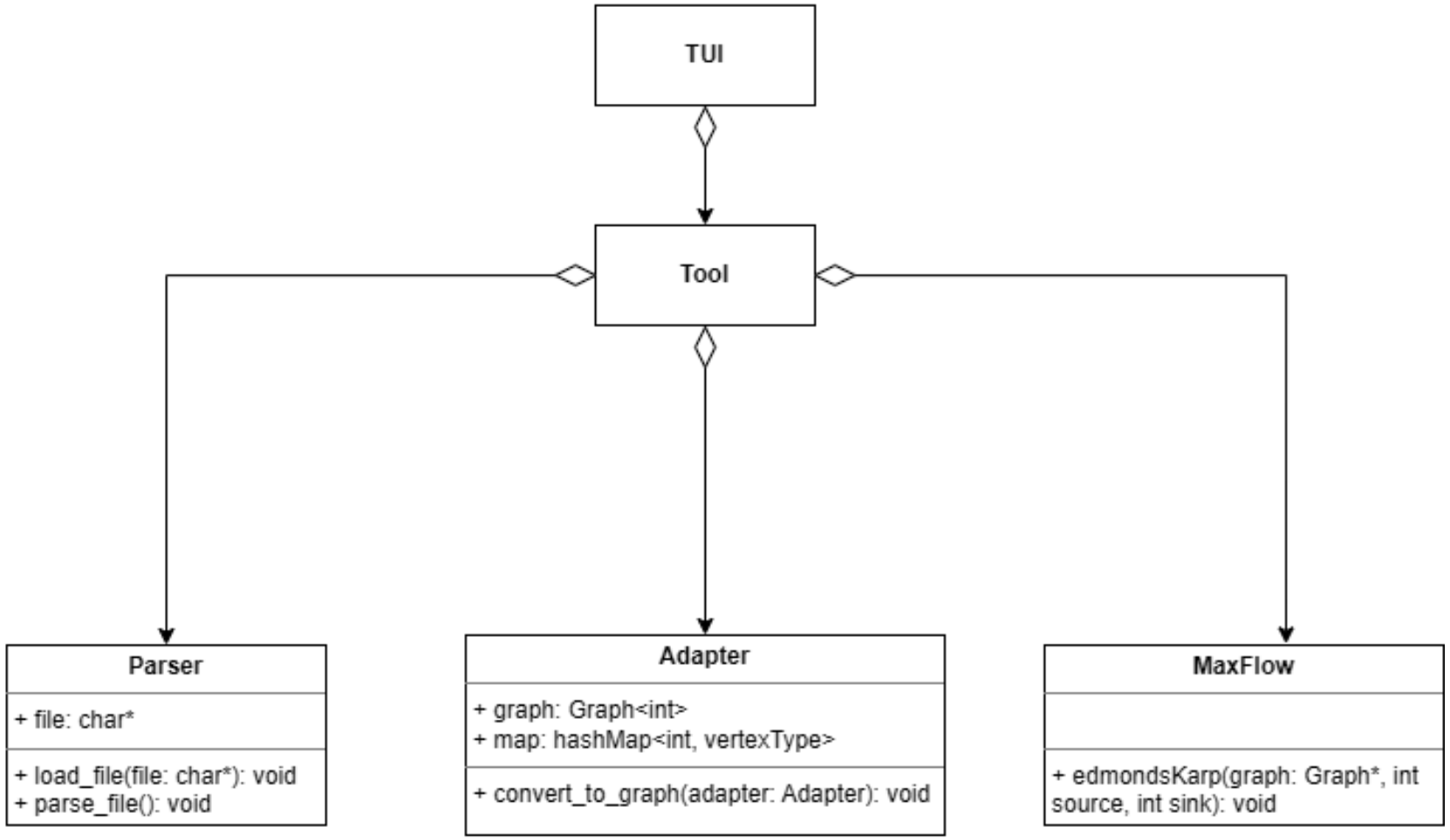
Scientific Conference Organization Tool

Description

- Set of articles with domains
- Set of reviewers with expertise in certain domains
- Find the best or one of the best distributions
- Reviewers can only review a maximum amount of articles
- An article needs at least a minimum amount of reviewers
- If not all articles can be assigned reviewers, the program prints the unassigned articles.
- Feature to find critical reviewers, that is: the reviewers that, if failing to do their assignment, will cause a failure.

Project structure

Scientific Conference Organization Tool



TUI

```
Please enter the option number.

Selected dataset: None

1: Description
2: Select Dataset
3: Run
4: Credits
5: Exit

Option:2
```

```
Enter dataset path:input/dataset1.csv
```

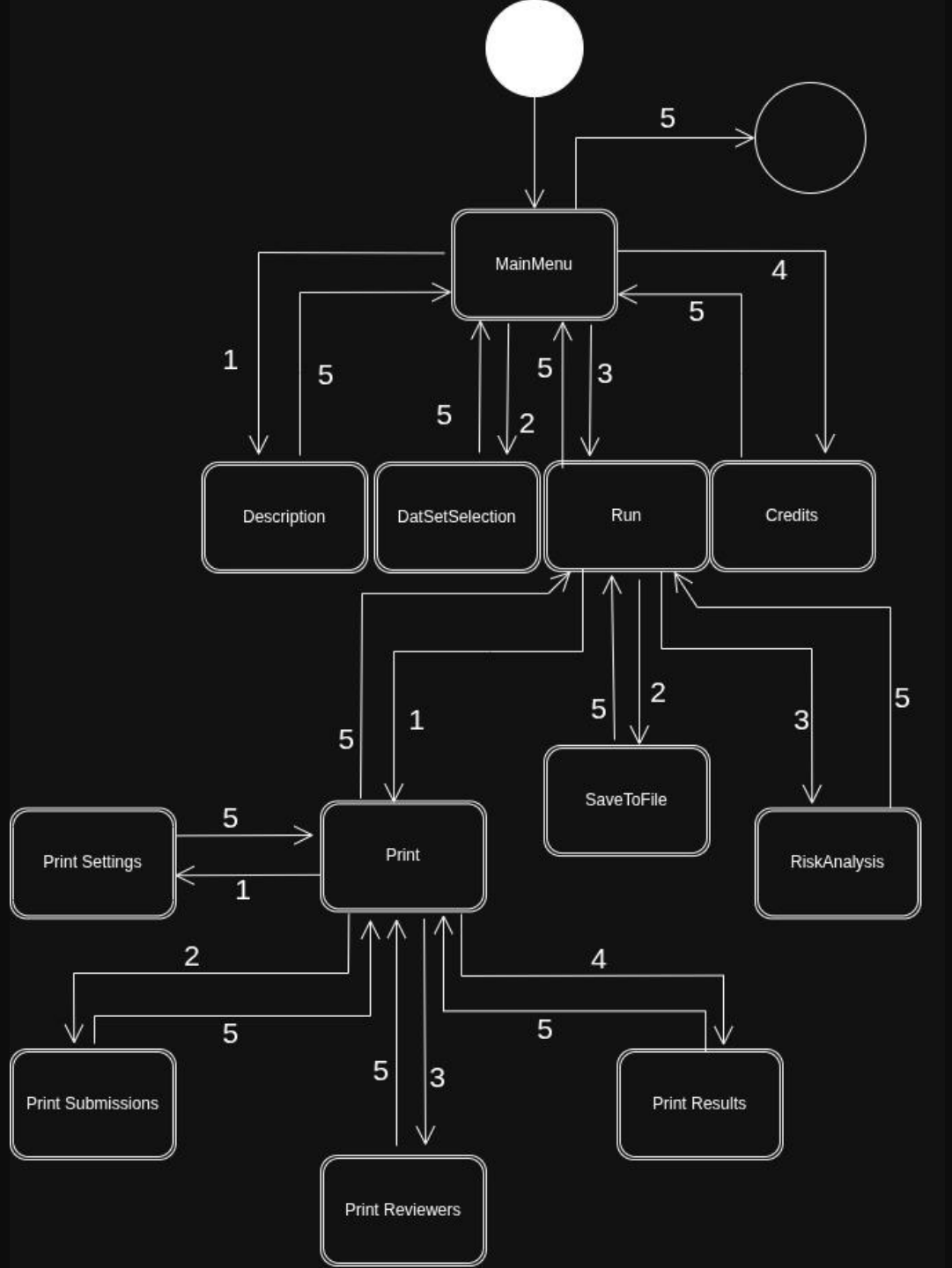
```
Please enter the option number.

Selected dataset: input/dataset1.csv

1: Description
2: Select Dataset
3: Run
4: Credits
5: Exit

Option:3
```

TUI walkthrough

- 1º Press 2 in the main menu to select the dataset
- 2º Write the relative path to the dataset
- 3º Press 3 to run

TUI

- Responsible for the terminal interface
- Handles user inputs
- Calls the respective tool methods for running the algorithms and showing outputs



TUI

How it works

The menuHandler() is the method responsible for creating the program_output folder, displaying the current menu and handling user input.
The getOption() method waits for user input, and then the optionHandler() method handles the chosen option.

```
void Tui::menuHandler() {
    isRunning = true;
    std::filesystem::create_directory("./program_output");

    while (isRunning) {
        displayMenu();
        int option = getOption();
        optionHandler(option);
    }
}
```

```
void Tui::run() {
    clear();
    if (isDataSetSelected()) {
        tool.setup(const_cast<char*>(dataSetPath.c_str()));
        tool.get_max_flow();
        tool.get_missing_output();

        runHandler();
```

The run() method is responsible for calling the max_flow algorithm implemented in the Tool class, and calling the runHandler() method that will determine the option that is chosen by the user.

TUI

How it works

runHandler()

Responsible for displaying the run menu and handling user input, that is, calling a specific method depending on what option the user chose.

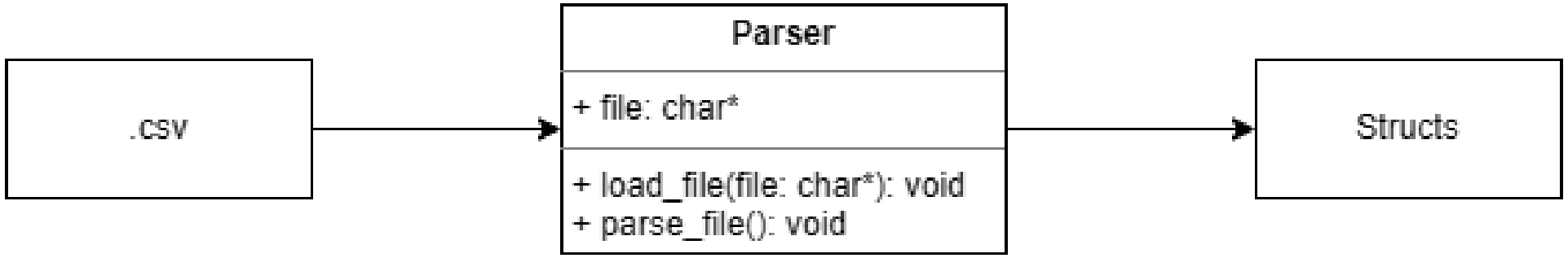
```
void Tui::runHandler() {
    while (true) {
        displayRunMenu();
        int option = getRunOption();
        if (option == 5) return;
        runOptionHandler(option);
    }
}
```

Options

1. Print
2. Save to file
3. Risk analysis

```
void Tui::runOptionHandler(int option) {
    switch (option) {
        case 1:
            printHandler();
            break;
        case 2:
            tool.save_to_file();
            onHold();
            break;
        case 3:
            tool.risk_analysis();
            tool.print_risk_analysis();
            onHold();
            break;
    }
```

Parser



Parser

How it works

- Responsible for reading and extracting structured data from the input file
- Input file is divided into clearly marked sections (#Submissions, #Reviewers, #Parameters, #Control)
- Parser reads each section to populate the corresponding internal data structures to be later handled by algorithms

Parser

Uses line by line reading and tokenization (std::strtok) to convert CSV input into structured objects

Helper functions handle string to integer conversion

- Trimming whitespace and quotes, and validating input values.

The main entry point, parse_file(), locates each section in the file and calls the appropriate parsing function

- Ensuring that all data is loaded and ready for further processing.

```
void Parser::parse_file() {
    std::ifstream in;

    in = find_header(file, "#Submissions");
    parse(in, submissions);

    in = find_header(file, "#Reviewers");
    parse(in, reviewers);

    in = find_header(file, "#Parameters");
    parse(in, parameters);

    in = find_header(file, "#Control");
    parse(in, control);
}
```

Parser

```
void Parser::parse_line(char* line, submission& s) {
    // Read Id
    s.id = read_value(std::strtok(line, ","));

    // Skipping title, authors and email
    std::strtok(NULL, ",");
    std::strtok(NULL, ",");
    std::strtok(NULL, ",");

    // Read primary
    s.primary = read_value(std::strtok(NULL, ","));

    // Read secondary
    s.secondary = read_value(std::strtok(NULL, ","));
}
```

Submissions

- To handle the submissions info, each line is tokenized to extract the submission ID and domain (primary and secondary).
- Non essential fields like title, authors, and email are ignored.
- Relevant information of valid submissions is stored in a submission structure, which is then stored in a vector of submissions.

Reviewers

- Each reviewer's ID and expertise domains are extracted similarly and stored in a reviewer structure and then in a vector.

Parameters

- When it comes to parameters, workflow parameters are read and then stored in a parameters_ structure.

Control

- Lastly, to handle control settings, control information is read, trimming any surrounding spaces or quotation marks and saving the relevant info in a control_ structure.

Parser

Responsibilities

01

Modular parsing for each data type (submission, reviewer, parameters, control)

02

Handling invalid entries adequately

03

Preparing all data for subsequent algorithms (Maximal Bipartite Matching, etc...) and menu-based interfaces

Parser

Why it needs to read the file from the start for each header

Some irregularities in the dataset file (e.g. An '#' between Submissions and Reviewers)

+

Some doubts about the .csv structure (e.g. Can the topics have a different order)

=

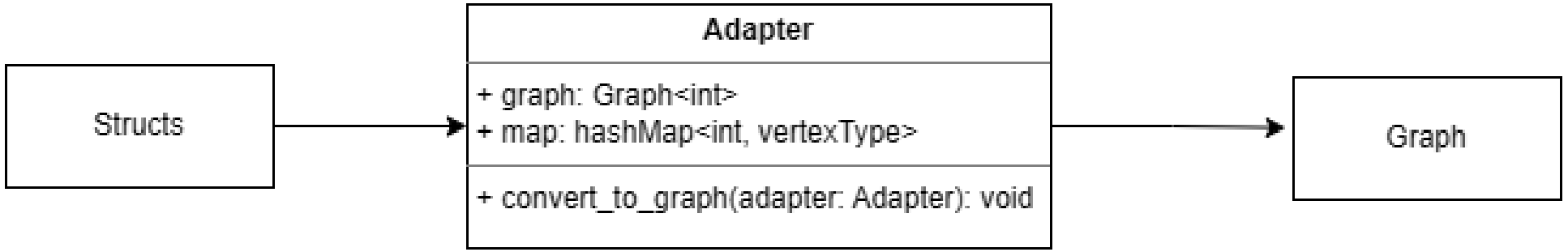
Create the most generic parser to avoid problems in the future.

Parser

Time complexity

- Let l be the number of lines and h be the number of headers, the upper bound can be defined to be O(l * h). So in this project, the time complexity to parse the dataset is O(4 * l) = O(l)

Adapter



Adapter

How it works

- Responsible for transforming the data processed by the parser into a graph structure
- The data in the parser is divided into 4 different attributes: reviewers, submissions, parameters and control.
- The adapter's goal is to create a bipartite graph which separates the reviewers and the submissions, to be later handled by a max flow algorithm.

Adapter

First of all, the adapter adds a source and a sink nodes to the graph, so it can be used by the max flow algorithm.

Then, it adds a node to the graph for each reviewer, adding an edge between the source node and each of the reviewer nodes, with weight equal to the maxReviews parameter.

It also creates an auxiliar map of each reviewer node id and the reviewers expertise, accounting on the parameters primaryRev and secondaryRev, that determine which of the domains are to have in account.

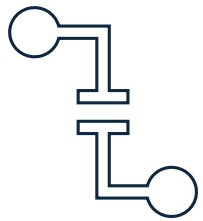
```
graph.addVertex(0);
graph.getVertexSet()[0]->id = -1;
map[0] = source;

graph.addVertex(1);
graph.getVertexSet()[1]->id = -2;
map[1] = sink;
```

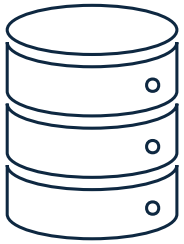
Adapter
How it works

```
enum vertexType {
    source,
    sink,
    submission,
    reviewer
};
```

After that, it adds a node to the graph for each submission, adding an edge between each of the reviewer nodes and the sink node, with weight equal to the minReviews parameter.

It also adds an edge between each reviewer node, and each submission node that has a common domain to have in account, using the reviewers expertise auxiliar map, with weight 1 because each reviewer can only review each submission once.





The parser has a map attribute, that maps every node to its type (source, sink, reviewer, submission), so that it is easier to read the graph's information after running the max flow algorithm, specially for showing the output.

Adapter

What it is responsible for

01

Converting parsed data into a bipartite graph structure (reviewers and submissions)

02

Assigning and managing node types (source, sink, reviewer, submission)

03

Establishing the appropriate connections between nodes for the max flow algorithm

04

Preparing the graph for subsequent algorithms (Max Flow, Risk analysis, etc…)

Adapter

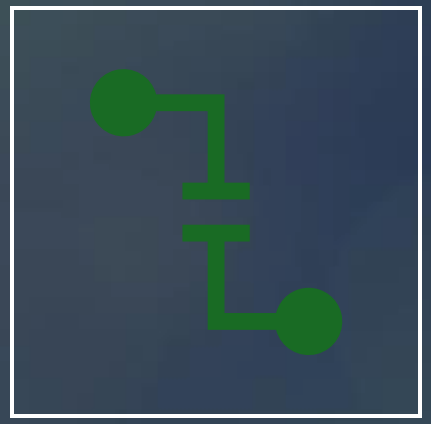
Time complexity

- Because it creates the graph, the time complexity is O(V + E)
- V: number of vertices
- E: number of edges

MaxFlow

How it works

Implements the Edmonds-Karp algorithm, an implementation of the Ford-Fulkerson method that uses a BFS, and, consequently, is shortest path "oriented", to compute the maximum flow in a directed graph.



This maximum flow is used to generate the assignments in the system by modeling the assignment problem as a bipartite graph

Max Flow – How it works

One set of vertices represents submissions and the other represents reviewers, and edges encode assignment possibilities with capacity constraints.

In this context, computing the maximum flow is equivalent to solving a maximal bipartite matching problem.

Each unit of flow along an edge represents an assignment of a reviewer to a submissiothe maximum flow guarantees that as many submissions as possible receive the required number of reviewers while respecting all reviewer and submission constraints.

MaxFlow

Time complexity

- Because it creates the graph, the time complexity is $O(V * E^2)$
- V: number of vertices
- E: number of edges

Tool

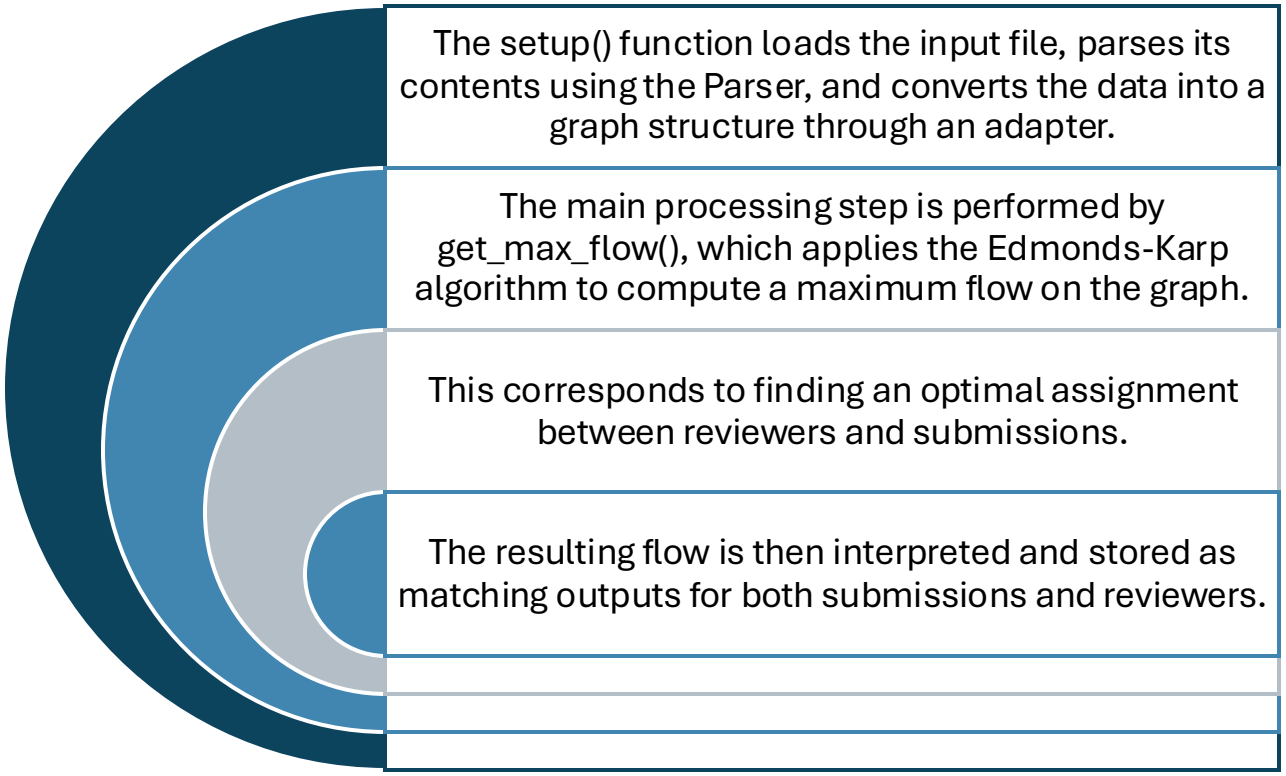
Tool

- How it works

- The Tool class is responsible for executing the core workflow of the system, meaning:
  - Transforming parsed input data into a graph representation,
  - Running the assignment algorithm, and the corresponding outputs.
- It is responsible for data preparation, algorithm execution, and result generation.

Tool

How it works




```
void Tool::setup(char* path) {
    this->path = path;
    parser.load_file(path);
    parser.parse_file();
    adapter.convert_to_graph(parser);
    is_setup_done = true;
}
```

```
void Tool::get_max_flow() {
    if (!path || !is_setup_done) {
        printf("Error: load setup dataset first\n");
        return;
    }

    max_flow.edmondsKarp(&adapter.graph, 0, 1);
    Graph<int> graph = adapter.graph;
```

Tool
How it works

The risk_analysis() function identifies critical reviewers whose removal would compromise the assignment. It performs a breadth-first search on the residual graph to determine which vertices are not reachable and derives the set of critical reviewers from this structure.

The get_missing_output() function identifies submissions that did not receive the required number of reviewers by inspecting edges whose flow does not meet their capacity. These are stored and sorted for reporting.

The class supports printing results to the console or saving them to a file. Outputs include the assignment results, missing assignments, and risk analysis.

Tool

What it is responsible for

01

Integrating parsing, graph construction, and algorithm execution

02

Computing assignments using maximum flow algorithm (Edmonds-Karp)

03

Detecting incomplete assignments and reports missing allocations

04

Perfoming a risk analysis for k=1

05

Supporting console output and file-based result storage

Risk Analysis

Risk analysis (k = 1)

- Risk analysis is the search for the set of reviewers in which are critical for the final result, i.e. if the reviewers in which if they fail to accomplish their work will result in one or more articles not having sufficient reviews.
- For k = 1, we need to find which reviewers alone could cause in failure.

Risk analysis (k = 1)
How it works

The key to find the critical reviewers is to analyse the residual graph.

If we traverse the residual graph (using the augmented paths) from a node, we will have two types of nodes: visited and unvisited

And the key for the success is in the unvisited ones that tells us that all paths which are able to reach that node are saturated and there is no other path to reach it that starts from the starting node.

That means if we remove one incoming edge from the node, the maximum flow would decrease.

With that idea in thought, we just need to traverse the graph from source through the residual graph, then we filter from all unvisited nodes the articles and we know that all reviewers who are reviewing the article are critical.

```
bfs(adapter.graph.getVertexSet()[0]);
Graph<int>* g = &adapter.graph;

for (size_t i = 0; i < g->getVertexSet().size(); i++) {
    Vertex<int>* v = g->getVertexSet()[i];
    if (v->isVisited()) continue;

    for (size_t j = 0; j < v->getIncoming().size(); j++) {
        if (adapter.map[v->getInfo()] == submission) {
            risk_output.insert(v->getIncoming()[j]->getOrig()->id);
        }
    }
}
```

Risk analysis (k = 1)

Time complexity

- The BFS takes O(V + E) and then we search for unvisited submissions and for their edges (O(V * E)). We can define our algorithm as O(V * E).
- But because the number of edges in this project is smaller or equal to 2, i.e. it is constant, the BFS is the part which takes more time.
- Overall: O(V + E)

Risk analysis (k > 1)

Pseudo-code

```
res = list()
list = generate_all_comb_of_reviewers(k)
for (comb in list)
    new_graph = graph - {reviewers in comb}
    if (max_flow(new_graph) < max_flow(graph)) add_to_res()

return res
```

Risk analysis (k > 1)

- Time complexity

- We would need to generate all combinations with k nodes, so we would need O(combinations of r choose k), let r be the number of reviewers.
- And for each combination, we would need to recalculate the maximum flow thus taking in the end O(combiations of r choose k * V * E^2)

Credits

This program was made by